Technical Requirements: Frontend Engineer

Owner: Fisayo Obadina | Project: SpendLens | Status: Draft | Date: 29 Oct 2026

1. Stack

- Next.js (App Router) with TypeScript, deployed on Vercel.
- Supabase JS client for login only. All data calls go to the FastAPI backend.
- Minimal dependencies. A plain fetch wrapper, no state library unless needed.

2. Environment variables

```
NEXT_PUBLIC_API_URL=
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
NEXT_PUBLIC_KORA_PUBLIC_KEY=      # test key (pk_test_...) until told otherwise
NEXT_PUBLIC_FIXTURE_MODE=false
```

No secret keys in the frontend. Only public and anon keys. Never commit .env.

3. Routes

Route	Purpose
/	Partner Bank host shell
/insights	Embedded SpendLens module home (summary or empty state)
/insights/import	Upload, preview, errors, confirm
/insights/review	Coverage, reconciliation candidates, category correction
/insights/plan	Protect essentials, scenario builder, explanation
/insights/pro	Pricing, checkout, payment status
/insights/forecast (v2)	Forecast and recurring candidates
/insights/chat (v3)	Chat panel

4. API client

Method and path	Purpose
GET /health	Service and database status
POST /v1/imports/preview	Validate a CSV, store nothing
POST /v1/imports	Commit a reviewed import (Idempotency-Key)
DELETE /v1/imports/{id}	Delete import and derived data
GET /v1/transactions	Paginated list with filters
PATCH /v1/transactions/{id}	Correct category or review state
POST /v1/reconciliation/confirm	Confirm or reject candidate links
GET /v1/analysis	Totals, coverage and warnings for a date range
PUT /v1/protections	Set protected categories
POST /v1/scenarios	Calculate a reduction scenario
POST /v1/scenarios/{id}/explanation	Grounded AI explanation (with fallback)
POST /v1/billing/orders	Create a Kora order (Idempotency-Key)
GET /v1/billing/orders/{id}	Order status for the frontend to poll
GET /v1/forecast (v2)	Category forecast with range or abstain
GET /v1/recurring (v2)	Recurring-payment candidates

Method and path	Purpose
POST /v1/chat (v3)	Streaming chat with read-only tools
DELETE /v1/me/data	Delete all of the user's financial data

- One file, lib/api.ts. Attach the Supabase access token as Authorization: Bearer.
- Send an Idempotency-Key (a fresh UUID per user action) on import commit and order creation. Reuse the same key when retrying the same action.
- In fixture mode, lib/api.ts returns JSON from /fixtures with the same shapes.
- Types: write by hand from the OpenAPI file, or generate if the team agrees to the extra tool.

5. Money display

```
const fmt = (kobo: number) =>
  new Intl.NumberFormat("en-NG", { style: "currency", currency: "NGN" }).format(kobo / 100);
```

Display only. Never add, subtract or multiply money values in the UI. Show the number the API sent.

6. Import wizard

- Client checks: extension .csv, size under 5 MB. Reject early with a message.
- POST to /v1/imports/preview. Show row count, date range, accounts found, and row-level errors with line numbers.
- User confirms. POST to /v1/imports with an idempotency key.
- Show coverage warnings from the API (for example "No transactions for 12 days in this range").

7. Review screen

- Reconciliation candidates: side-by-side rows, buttons "Same payment" and "Different payments", sent to /v1/reconciliation/confirm.
- Category correction: inline select, saved with PATCH, results refresh.
- Label uncategorized rows as "Needs review".

8. Scenario builder

- Essentials toggles call PUT /v1/protections. Protected categories are disabled in the builder.
- Reduction input: whole-number percent per category. POST /v1/scenarios.
- Show baseline, reduction, result, per-category breakdown, link to supporting transactions, and the label "Scenario, not realized savings".
- Explanation: POST /v1/scenarios/{id}/explanation. If it fails or times out, show the calculation without it and a short notice.

9. Kora checkout

Follow Kora's Checkout Standard documentation.

```
// 1. POST /v1/billing/orders  -> { id, reference, amount, currency, notification_url }
// 2. Open checkout with values FROM THE ORDER, not typed in the client
window.Korapay.initialize({
  key: process.env.NEXT_PUBLIC_KORA_PUBLIC_KEY,
  reference: order.reference,
  amount: order.amount,
  currency: order.currency,
  customer: { name, email },
  notification_url: order.notification_url
});
// 3. Poll GET /v1/billing/orders/{id} every 3 seconds, up to 2 minutes
```

```
//    Unlock Pro only when status === "paid"
```

- Load Kora's script only on the Pro page.
- The docs the team read do not state whether the amount is in naira or kobo. Confirm in the sandbox with Olamilekan before hardcoding anything. The backend owns the conversion.
- Use any success or close callbacks from Kora's docs for UX only (for example to start polling). They never unlock features.
- States to design: pending, paid, underpaid, overpaid, failed, cancelled by user, timed out.

10. Forecast and chat (v2, v3)

- Forecast card: point value, low and high range, method label, and a "Not enough data yet" state when abstained is true.
- Chat: fetch to /v1/chat and read the streamed response. Render evidence chips that link to transactions. Show a stop button and an error state.
- Voice (stretch): browser SpeechRecognition API if present, feature-detected. Support varies [unsure], so text input stays the default. Do not send audio to our servers.

11. Accessibility and quality checklist

- Semantic HTML, labeled inputs, visible focus, keyboard-operable tables and dialogs.
- Contrast AA. Status never conveyed by color alone.
- Respect reduced-motion preference. No decorative animation.
- Responsive down to 360px width.
- Loading, empty and error states for every screen.

12. Tests

- Manual script covering import, correction, scenario, payment states, delete data.
- One smoke test (if time allows) that loads the app in fixture mode and completes the flow.

13. Security

- No raw CSV contents in the browser console, analytics, or error trackers.
- Render transaction descriptions as text, never as HTML.
- Session handled by Supabase. Log out clears local state.